GRADUATION PROJECT · SYSTEM ANALYSIS & DESIGN · MEMBER 3

SmartListing Copilot — System Architecture & Component Design

27 October 2026 · Habiba · Status: design proposal, team review pending

1. Purpose, Scope and Shared Terminology

This document defines the architecture of SmartListing Copilot: which components exist, what each one is responsible for, and how they exchange data. It builds on the completed Requirements phase and the Revised Scope; it does not rewrite any requirement.

In scope for the architecture: one listing (image, title, description, seller category) goes through parallel Computer Vision and NLP analysis, then comparison, scoring, suggestion and a mandatory seller decision. The system covers five fashion categories (selected after EDA), and only the attributes Product Type and Color.

Out of scope (not drawn anywhere in this document): LLM components, return prediction, dynamic pricing, price validation, live marketplace integration, production cloud deployment, batch CSV processing and automatic publishing.

> **Status note.** This is a design proposal. No component is claimed to be implemented or evaluated, and no performance figure is stated.

Canonical component names

These names are used in every diagram and table below. All members should reuse them exactly.

Canonical name	Short description
Streamlit Frontend	Seller-facing UI
FastAPI Backend	API layer and analysis orchestrator
CV Module	Visual Product Type and Color prediction
NLP Module	Product Type and Color claim extraction from text
Mismatch Detection Module	Compares CV evidence with NLP claims; reason codes and severity
Scoring & Triage Module	Listing Quality Score, Listing Inconsistency Level, Review Queue state
Suggestion Module	Template-based Before/After corrections
Database	Persistent storage of listings, results and seller decisions
Model & Lexicon Store	Trained weights, thresholds, color/type vocabularies, Arabic lexicon
Shared Schemas	Common Pydantic data contract between modules

Requirement IDs (FR-001 to FR-025) refer to the Functional Requirements Specification. Its terms map to the names above: Streamlit UI = Streamlit Frontend; Comparison engine = Mismatch Detection Module; Scoring module and Review/scope engine = Scoring & Triage Module; Generation module = Suggestion Module. Non-functional requirements (NFR-PERF, SEC, REL, USA, MNT, TEC) and use cases (UC-01 to UC-13) from the Non-Functional Requirements and Use Cases document are cited where a component realizes them.

2. System Architecture

The system has four layers: a Streamlit Frontend, a FastAPI Backend that orchestrates each request, three analysis modules, and storage. The CV and NLP modules run in parallel, and nothing reaches a marketplace without the seller's decision.

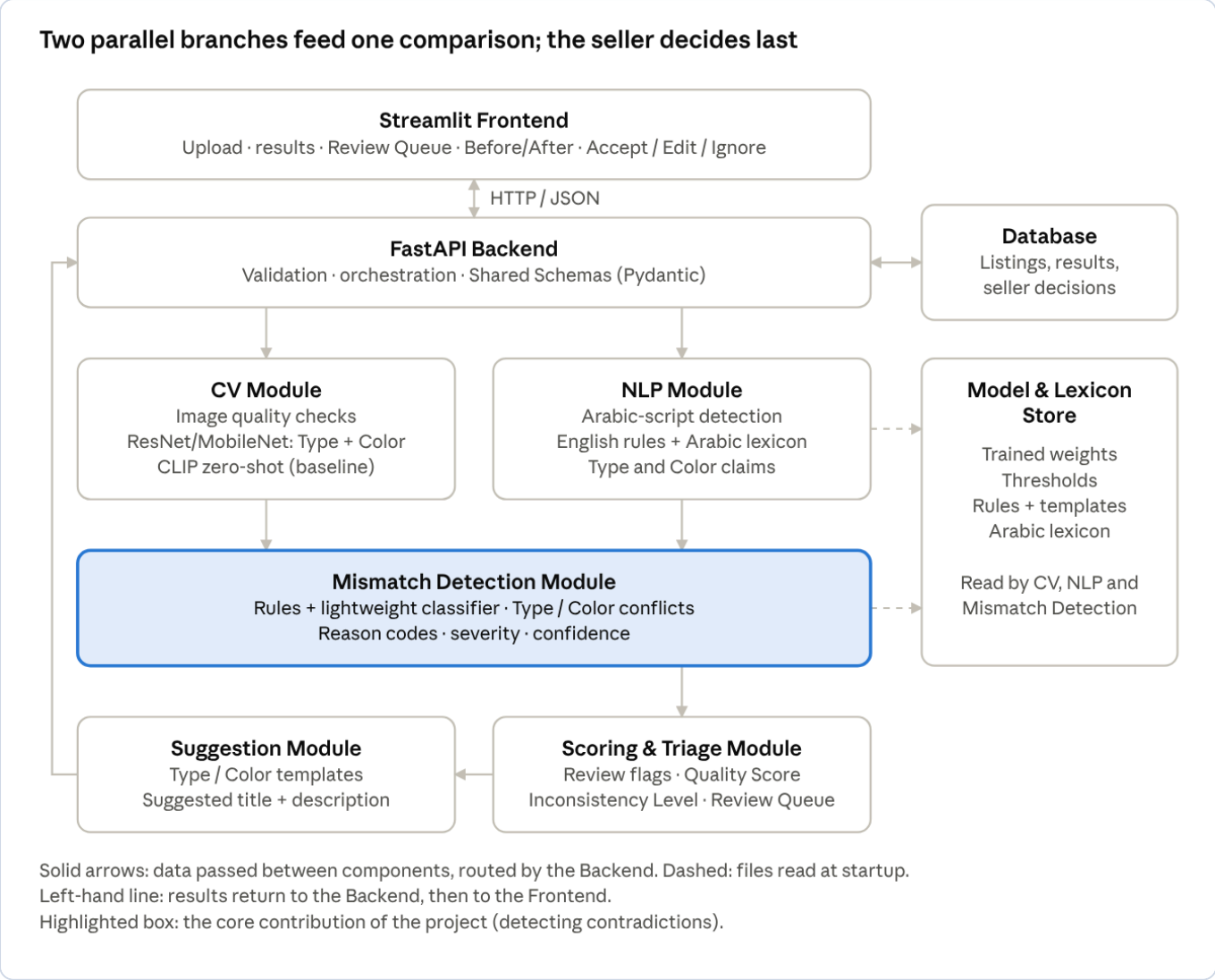


Figure 1 — System architecture · 10 components

The Frontend talks only to the Backend. The Backend sends the listing to the CV Module and the NLP Module at the same time. The Mismatch Detection Module compares their outputs, then Scoring & Triage and the Suggestion Module prepare what the seller sees. The Backend stores results and seller decisions in the Database.

3. Component Diagram

The Backend is the only component that calls the analysis modules. Each module receives and returns a typed object and never calls another module, so any module can be replaced by a stub during integration.

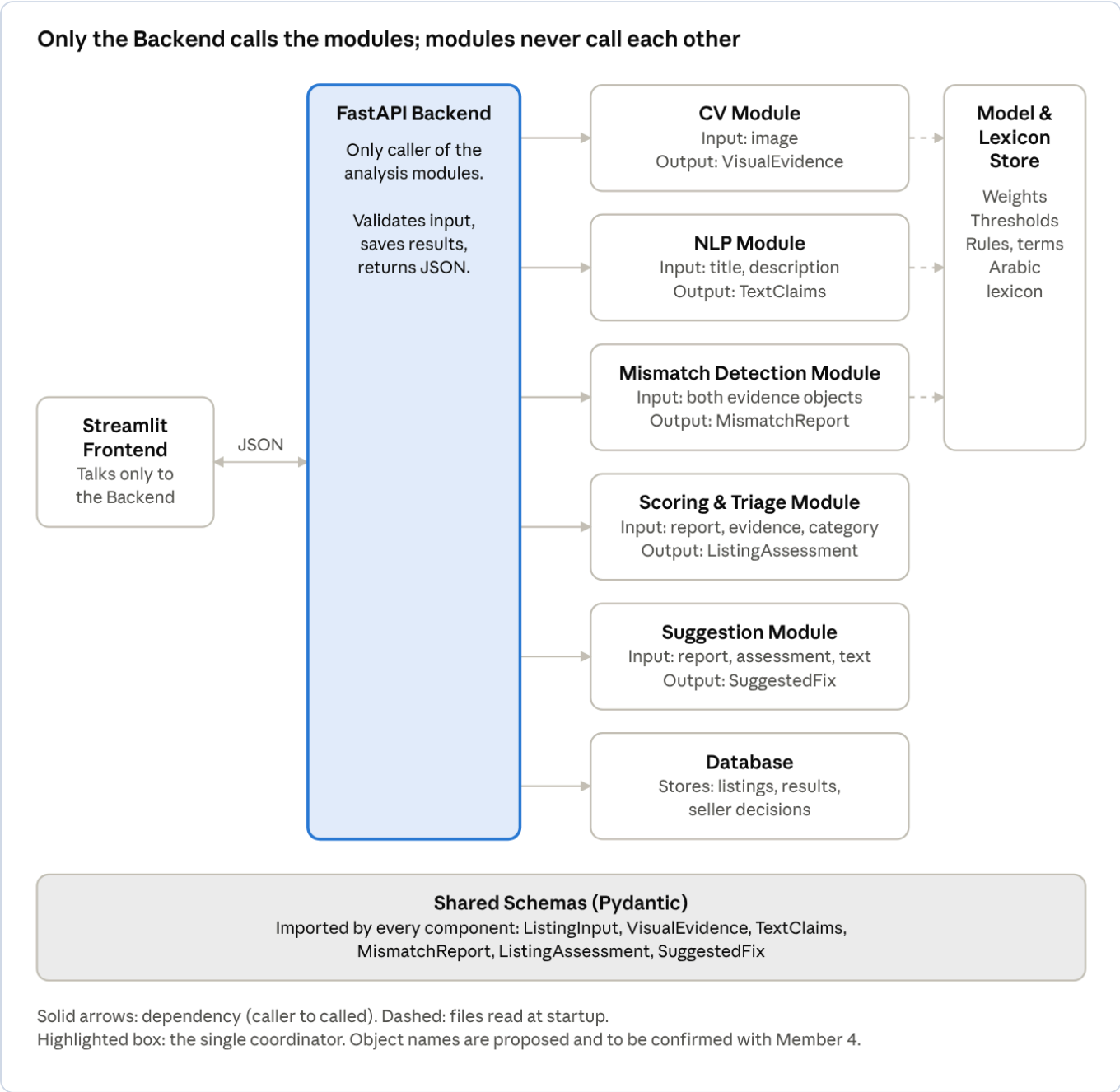


Figure 2 — Component diagram · 9 components + shared schemas

The CV, NLP and Mismatch Detection modules read weights, thresholds and vocabularies from the Model & Lexicon Store at startup. Every component imports the same Shared Schemas, which is what keeps the hand-offs consistent.

4. Component Responsibilities

Each component has one job and a defined input and output. The table is the reference other members should use when naming components or writing API schemas.

Component	Responsibility	Input	Output	Depends on	Requirements realized
Streamlit Frontend	Collects image, title, description and category; shows predicted vs claimed attributes, confidence, reason codes, severity and Review Queue state; shows original and suggested text side by side with the reason for each change; records the seller's Accept / Edit / Ignore choice; shows the Review Queue grouped by state for the Catalog Reviewer; displays at most one warning per reason code per attribute, most severe first; states that Arabic input is limited to the fixed lexicon	Seller actions	ListingInput, SellerDecision	FastAPI Backend	FR-001 to FR-003, FR-022 to FR-025; displays the results of FR-013, FR-016, FR-017, FR-019, FR-020; NFR-USA-02 to NFR-USA-09; UC-13 (Review Queue view)
FastAPI Backend	Validates the request, calls CV and NLP in parallel, then each later module in order, passes typed objects between them, saves results and returns JSON; logs the time of each stage; keeps working if one branch fails and reports the listing as Needs review with an explanation; never shows stack traces or file paths to the user; has no publishing path	HTTP requests	HTTP responses	All modules, Database, Shared Schemas	Workflow in FRS section 9; input rules IN-01 to IN-04 are still open; NFR-PERF-01, 02, 09; NFR-REL-04; NFR-SEC-01 to 04, 07
CV Module	Checks image resolution, blur and brightness; predicts Product Type within the five selected classes and Color (with a dominant-color cross-check) using the ResNet/MobileNet model; runs CLIP zero-shot as baseline and fallback and exposes whether CV and CLIP agree; switches to CLIP or escalates to review if the primary model is unavailable; loads weights once at startup	Product image	VisualEvidence	Model & Lexicon Store	FR-004, FR-005, FR-006; visual part of FR-013; NFR-REL-02, NFR-PERF-08
NLP Module	Detects Arabic script to choose the fixed Arabic lexicon (otherwise English); normalizes text; extracts only Product Type and Color claims with a controlled vocabulary, rules and fuzzy matching	Title, description	TextClaims	Model & Lexicon Store	FR-007, FR-008
Mismatch Detection Module	Compares VisualEvidence with TextClaims using rules and a lightweight classifier; labels each attribute consistent, contradictory or unclaimed; raises TYPE_MISMATCH, COLOR_MISMATCH and (if kept) MISSING_ATTRIBUTE; sets severity from the rule table and attaches confidence and an explanation	VisualEvidence, TextClaims	MismatchReport	Model & Lexicon Store	FR-009 to FR-012, FR-016, FR-017; finding confidence of FR-013
Scoring & Triage Module	Raises review flags LOW_CONFIDENCE (below the validation-derived threshold), IMAGE_BLUR and UNSUPPORTED_CATEGORY (CV/CLIP disagreement, category contradiction, other-product preference); computes the Listing Inconsistency Level and the optional Listing Quality Score; assigns the Review Queue state	MismatchReport, VisualEvidence, seller category	ListingAssessment	Model & Lexicon Store (threshold), Shared Schemas	FR-003 (category use), FR-014, FR-015, FR-018, FR-019, FR-020; reason codes of FR-016 for flags; same input always gives the same result (NFR-REL-03); UC-05, UC-07, UC-08

Component	Responsibility	Input	Output	Depends on	Requirements realized
Suggestion Module	Builds the suggested title and description from controlled templates, using only supported Type and Color; leaves out attributes under low confidence or out-of-scope flags; records what changed and why; no LLM	MismatchReport, ListingAssessment, original text	SuggestedFix	Shared Schemas	FR-021 (supplies the data for FR-022)
Database	Stores listings, analysis results and seller decisions, so the Catalog Reviewer can open the queue grouped by state	Records from the Backend	Stored records	None	Seller decisions of FR-023 to FR-025 (how they are stored is still open); queue retrieval for UC-13
Model & Lexicon Store	Holds trained weights, validation-frozen thresholds (confidence, blur, brightness, resolution), color/type vocabularies, the Arabic lexicon, the type-group and color-family tables, and the reason-code and suggestion templates, all as configuration rather than code	None	Files read at startup	None	Threshold of FR-014; vocabularies of FR-007 and FR-008; rule tables of FR-017; NFR-MNT-04, NFR-MNT-05
Shared Schemas	Defines the typed objects exchanged between components	None	Pydantic models	None	Integration boundary in FRS section 11

5. Communication and Data Exchange

The Frontend and Backend talk over HTTP with JSON bodies. Inside the Backend, the modules are called as Python functions that take and return Pydantic objects, so no network hop sits between modules in the MVP.

Main analysis flow

1. The seller enters the image, title, description and seller category in the Frontend, which sends one request to the Backend.
2. The Backend validates the request. A missing image stops the analysis (confirmed in the FRS). The rules for empty titles or descriptions, unreadable images and malformed categories are still open (IN-01 to IN-04), so any rejection returns a defined error and no module is called.
3. The Backend calls the CV Module and the NLP Module in parallel, as the FRS workflow specifies.
4. The Backend passes both outputs to the Mismatch Detection Module and receives a MismatchReport.
5. The Backend passes the MismatchReport, the VisualEvidence and the seller category to the Scoring & Triage Module and receives a ListingAssessment.
6. The Backend passes the MismatchReport, the ListingAssessment and the original text to the Suggestion Module and receives a SuggestedFix.
7. The Backend saves the listing and results in the Database and returns one combined response.
8. The Frontend shows the results. The seller chooses Accept, Edit or Ignore, and the Backend stores that decision. Nothing is published automatically.

Data objects exchanged

Field lists are a proposal for discussion. The final names and types belong to the API Specification (Member 4) and the Data Dictionary (Member 2).

Object	Produced by	Main fields
ListingInput	Frontend	image, title, description, seller_category
VisualEvidence	CV Module	predicted type and color with confidence, image quality findings (resolution, blur, brightness), CV/CLIP agreement signal, model used (transfer-learning or CLIP zero-shot)
TextClaims	NLP Module	claimed type, claimed color, script route (Arabic lexicon or English), matched text span
MismatchReport	Mismatch Detection Module	findings (reason code, severity, confidence, explanation, predicted vs claimed value), missing attributes
ListingAssessment	Scoring & Triage Module	review flags with reason codes, Listing Inconsistency Level (Low / Medium / High), optional Listing Quality Score (0 to 100), Review Queue state
SuggestedFix	Suggestion Module	original text, suggested text, changed attributes with the reason for each change
SellerDecision	Frontend	action (Accept / Edit / Ignore), edited text when the action is Edit

Failure handling

Invalid input, such as a missing image, stops the analysis with a defined, non-technical error and no module is called. If one branch fails (for example NLP), the Backend keeps the pipeline running and passes the failure to the Scoring & Triage Module, which marks the listing Needs review with an explanation, so a failed

analysis is never shown as clean. Low confidence, blur and unsupported categories also route to Needs review (LOW_CONFIDENCE, IMAGE_BLUR, UNSUPPORTED_CATEGORY). Errors never show stack traces or file paths. The exact error codes are defined with the API Specification.

6. Design Decisions, Constraints and Assumptions

The architecture favors a simple, explainable structure that a team of six can build and integrate by the implementation milestone. Each decision below states what was chosen, why, and what it costs.

Decision	Why	Trade-off accepted
The Backend is the only caller of the modules	Modules stay independent, so each can be built, tested and replaced by a stub on its own	Workflow logic sits in the Backend
CV and NLP analyze the listing separately	Image evidence and text claims stay distinct, which makes every warning explainable	No joint image-text model; a full multimodal transformer is out of scope
Comparison happens after both branches finish (decision level)	The Mismatch Detection Module can show predicted vs claimed values and a reason code	The comparison cannot learn cross-modal features
Rules first, lightweight classifier second	Rule output is easy to explain; the classifier is tested against the rule baseline	Classifier training data for mismatches is synthetic
CLIP zero-shot lives inside the CV Module as baseline and fallback	It is compared with the transfer-learning model, not used as a preprocessing step	A fallback rule (when it is used) still has to be defined
Modules run as Python code inside one FastAPI application	Simplest to build and demo; production cloud deployment is out of scope	No independent scaling of modules
Shared Pydantic Schemas define every hand-off	One contract lets members build against stubs early (NFR-MNT-02, 03)	Schema changes need agreement across members
Rules, thresholds and templates are configuration, not code	A rule or threshold can change without editing logic (NFR-MNT-04, 05); thresholds are frozen on validation data, not tuned on the held-out manual set	The Store must be loaded and validated at startup
Quality Score and Missing Attribute Detection sit behind configuration flags	They can be cut under the fallback plan without breaking the core (NFR-MNT-06)	Extra branches in Scoring & Triage and Mismatch Detection
Decision logic is deterministic	The same model outputs always give the same severity, level, queue state and suggestion (NFR-REL-03)	Model inference must be fixed (evaluation mode, no sampling)
The prototype runs locally with no public endpoint and no external calls	Uploaded content stays on the machine (NFR-SEC-06, 07)	Weights must be downloaded beforehand
No publishing path exists in the system	The Backend stores the seller's decision and never calls a marketplace (NFR-SEC-01)	Suggested fixes are only suggestions

Constraints from the project scope

- Five fashion categories, selected after EDA; the final list is not fixed yet.
- Only Product Type and Color are checked.
- NLP is English-first with a small fixed Arabic lexicon.
- No LLM, return prediction, dynamic pricing or price validation.

- No user accounts, administrator role, authorization layer or admin dashboard; the prototype runs locally with no public endpoint.
- One listing per request; batch CSV processing is out of scope.
- Fashion Product Images Small is the only dataset.
- The Review Queue state for a lone Major mismatch (Q-01) and for conflicting flags (Q-02) is undecided. Queue routing is kept in the Scoring & Triage Module only, so the approved rule can be applied in one place.

Assumptions to verify

- Hardware for training and inference has not been checked, so latency is not specified here.
- The final five classes, class groups, color-family tolerance and confidence threshold come from EDA and validation, and are not fixed in this document.
- Optional items (MLflow logging, Docker packaging, CLIP linear probe) are not drawn because they are not part of the core workflow.

7. Consistency Notes for Other Members

These points keep the diagrams, database, API and UI aligned. Component names in this document are the canonical ones.

Member	What to align with this document
1 (UML)	Use the canonical component names as lifelines and actors. The call order is the eight steps in section 5; CV and NLP run in parallel. UC-13 (the reviewer opens the queue) needs the Database and a queue view in the Frontend.
2 (Database)	The Database stores listings, the results of each module (VisualEvidence, TextClaims, MismatchReport, ListingAssessment, SuggestedFix) and seller decisions. Model weights, lexicons and rule tables stay in the Model & Lexicon Store, not in the Database. The Database must also support listing stored results grouped by Review Queue state (UC-13).
4 (API)	Endpoints must cover submitting a listing, retrieving the result, recording a seller decision, and listing stored results grouped by Review Queue state (UC-13). Use the objects in section 5 and add the error cases (missing fields, unsupported image format, failed analysis).
5 (UI/UX)	Screens map to the Streamlit Frontend responsibilities: input, results with reason codes, Review Queue state, Before/After view, Accept / Edit / Ignore.
6 (Validation)	Section 4 lists the FR IDs each component realizes. A few requirements (FR-003, FR-013, FR-016) span two components. Section 4 also cites the non-functional requirements each component realizes. NFR-PERF-04 to 07, NFR-REL-05, NFR-REL-06 and NFR-SEC-05 concern evaluation and data discipline, not a runtime component, so they are not mapped here.

Open questions

- Which database engine will be used? To be agreed with Member 2.
- How is the image sent to the Backend (multipart upload or encoded in JSON)? To be decided in the API Specification.
- When does the CLIP fallback replace the transfer-learning prediction? To be defined with the CV work.
- Q-01 and Q-02 (Review Queue for a lone Major mismatch and for conflicting flags) are waiting for a team or supervisor decision. They change the routing rules in the Scoring & Triage Module.
- IN-01 to IN-04 (empty title or description, unreadable image, malformed category) are open. They change the Backend validation and the API error responses.

- How, and for how long, are seller decisions stored? The FRS leaves this open, and it affects the Database.
- UC-13: the MVP has no reviewer accounts, so the queue view is assumed to be a page in the same Streamlit app with no login. To be confirmed with Member 5 (UI/UX).